AIM: Write a program for matrix Manipulation using array.

OBJECTIVES: To study matrix manipulation using array.

OUTCOMES: The students will be able to perform basic matrix operations using array.

INPUT: Matrix elements

OUTPUT:

a) Matrix addition

b) Matrix multiplication

c) Transpose matrix

THEORY:

In matrix manipulation we perform different operations on matrix such addition, subtraction, multiplication, transpose of matrix etc. Rules for performing different operations are as follows:

Addition

- Order of the matrices must be the same
- Add corresponding elements together
- Matrix addition is commutative
- Matrix addition is associative

Subtraction

- The order of the matrices must be the same
- Subtract corresponding elements
- Matrix subtraction is not commutative (neither is subtraction of real numbers)
- Matrix subtraction is not associative (neither is subtraction of real numbers)

Scalar Multiplication

A scalar is a number, not a matrix.

- The matrix can be any order
- Multiply all elements in the matrix by the scalar
- Scalar multiplication is commutative
- Scalar multiplication is associative

PROGRAM:

```
/* To perform matrix manipulation */
#include<stdio.h>
#include<conio.h>
void main()
{
        int i,j,k,a[3][3],b[3][3],c[3][3];
        clrscr();
        printf("Enter 1st matrix element");
        for(i=0;i<3;i++)
        {
                for(j=0;j<3;j++)
                {
                        scanf("%d",&a[i][j]);
                }
    }
    for(i=0;i<3;i++)
    {
            for(j=0;j<3;j++)
            {
                    printf("\t%d",a[i][j]);
            }
            printf("\n");
  }
  printf("Enter 2nd matrix element");
  for(i=0;i<3;i++)
  {
          for(j=0;j<3;j++)
          {
                  scanf("%d",&b[i][j]);
          }
}
for(i=0;i<3;i++)
{
```

```
        for(j=0;j<3;j++)
        {
            printf("\t%d",b[i][j]);
        }
 printf("\n");
 }
printf("\n Matrix add\n");
for(i=0;i<3;i++)
      {
          for(j=0;j<3;j++)
          {
              c[i][j]=a[i][j]+b[i][j];
          }
    }
    for(i=0;i<3;i++)
    {
         for(j=0;j<3;j++)
         {
             printf("\t%d",c[i][j]);
         }
         printf("\n");
   }
   printf("\n matrix multi\n");

   for(k=0;k<3;k++)
   {
        for(j=0;j<3;j++)
        {
            c[k][j]=0;
            for(i=0;i<3;i++)
            {
                c[k][j]+=a[k][i]*b[i][j];
            }
        }
}
```

```
        for(k=0;k<3;k++)
        {
                for(j=0;j<3;j++)
                {
                        printf("\t%d",c[k][j]);
                }
                printf("\n");
        }
        printf("\ntranspose matrix\n");
        for(i=0;i<3;i++)
        {
                for(j=0;j<3;j++)
                {
                        printf("\t%d",a[j][i]);
                }
        printf("\n");
        }
    getch();
}
```

OUTPUT:

Enter 1st matrix element 1 2 3 3 2 1 4 2 1

1 2 3

3 2 1

4 2 1

Enter 2nd matrix element 1 2 3 3 2 1 4 2 1

1 2 3

3 2 1

4 2 1

Matrix add

2	4	6
6	4	2
8	4	2

matrix multi

19	12	8
13	12	12
14	14	15

transpose matrix

1	3	4
2	2	2
3	1	1

CONCLUSION: We have studied different matrix manipulation using array.

Experiment No. : 2

Implement Stack using array

AIM: Implement Stack using array.

OBJECTIVES:

OUTCOMES: Students will be introduce stack data structure and its operations.

INPUT: Enter numbers.

OUTPUT: Using choices elements are added and deleted from stack.

THEORY:

A stack is an abstract data type (ADT), commonly used in most programming languages. It is named stack as it behaves like a real-world stack, for example – deck of cards or pile of plates etc.

A real-world stack allows operations at one end only. For example, we can place or remove a card or plate from top of the stack only. Likewise, Stack ADT allows all data operations at one end only. At any given time, We can only access the top element of a stack.

This feature makes it LIFO data structure. LIFO stands for Last-in-first-out. Here, the element which is placed (inserted or added) last, is accessed first. In stack terminology, insertion operation is called **PUSH** operation and removal operation is called **POP** operation.

Algorithm/Pseudo code:

Algorithm for PUSH Operation:

Step 1. begin procedure push: stack, data

Step 2. if stack is full

Step 3. returnnull

Step 4. endif

Step 5. top ← top +1

Step 6. stack[top]← data

Step 7. end procedure

Algorithm for POP Operation:

Step 1. begin procedure pop: stack

Step 2. if stack is empty

Step 3. return null

Step 4. end if

Step 5. data ← stack[top]
Step 6. top ← top - 1
Step 7. return data
Step 8. end procedure

PROGRAM:

```
/* To implement stack using array */

#include<stdio.h>
void PUSH();
void POP();
void Search();
void Disp();

int stack[5],top,no,i,ch;

void main()
{

clrscr();

top=-1;   // initially stak is empty
printf("1.PUSH  \n2.POP  \n3.Search  \n4.Exit");
do
{
      printf("\n\nEnter your choice : ");
      scanf("%d",&ch);
      switch(ch)
      {
            case 1: PUSH();              break;
            case 2: POP();         break;
            case 3: Search();      break;
            case 4: return;
            default : printf("Wrong choice ");
```

```
        }
    }while(ch!=5);
    }

    void PUSH()
    {
        if(top==4)   // stack is full
            printf("Stack overflow");
        else
        {
            printf("Enter no: ");
            scanf("%d",&no);
            top++;
            stack[top]= no;
            Disp();
        }
    }

    void POP()
    {
        if(top==-1)
            printf("Stack is empty ");
        else
        {
            printf("POPed number : %d\n",stack[top]);
            top--;
            Disp();
        }
}

void Search()
{
    if(top==-1)
        printf("Stack is empty");
    else
```

```
        {
                printf("Enter a no. to search : ");
                scanf("%d",&no);
                Disp();
                for(i=0; i<=4; i++)
                {
                        if(stack[i]==no)
                        {
                                printf("\nNumber is at location : %d",i);
                                return;
                        }
                }
                printf("\nNumber not present");
        }
}


void Disp()
{
        if(top==-1)
                printf("Stack is empty ");
        else
        {       printf("Stack : Top->");
                for(i=top;i>=0;i--)
                        printf("\t%d \n\t",stack[i]);
        }
}
```

OUTPUT:

1.PUSH

2.POP

3.Search

4.Exit

```
Enter your choice : 1
Enter no: 11
Stack : Top->  11


Enter your choice : 1
Enter no: 12
Stack : Top->  12
               11


Enter your choice : 2
POPed number : 12
Stack : Top->  11


Enter your choice : 3
Enter a no. to search : 11
Stack : Top->  11

Number is at location : 0

Enter your choice : 4  */
```

CONCLUSION: In this way we have studied push and pop operations on stack.

Experiment No. : 3

Implement linear queue using array

AIM: Implement linear queue using array.

OBJECTIVES: Students will introduce queue.

OUTCOMES: Students will be able to implement queue

INPUT: Enter elements

OUTPUT: Using choices queue operations are performed

THEORY:

Queue is an abstract data structure, somewhat similar to Stack. In contrast to Queue, queue is opened at both end. One end is always used to insert data (enqueue) and the other is used to remove data (dequeue). Queue follows First-In-First-Out methodology, i.e., the data item stored first will be accessed first.

A real world example of queue can be a single-lane one-way road, where the vehicle enters first, exits first. More real-world example can be seen as queues at ticket windows & bus-stops.

Algorithm/Pseudo code:

Algorithm to insert element in Queue:

Step 1. If (REAR = size) then //Queue is full
Step 2. print "Queue is full"
Step 3. Exit
Step 4. Else
Step 5. If (FRONT = 0) and (REAR = 0) then //Queue is empty
Step 6. FRONT = 1
Step 7. End if
Step 8. REAR = REAR + 1 // increment REAR
Step 9. Que[REAR] = ITEM
Step 10. 10. End if
Step 11. Stop

Algorithm to delete element in Queue:

Step 1. If (FRONT = 0) then
Step 2. print "Queue is empty"
Step 3. Exit
Step 4. Else
Step 5. ITEM = Que [FRONT]
Step 6. If (FRONT = REAR)
Step 7. REAR = 0
Step 8. FRONT = 0
Step 9. Else
Step 10. FRONT = FRONT + 1
Step 11. End if
Step 12. End if
Step 13. Stop

PROGRAM:

```
/* Simple Queue */
#include<stdio.h>
void Insert();
void Delete();
void Disp();

int queue[5];
int front,rear,i,ch,num;

void main()
{
clrscr();
front=rear=-1;    // initially queue is empty

printf("1.Insert \n2.Delete \n3.Diplay \n4.Exit");
do
{
        printf("\n\nEnter choice : ");
        scanf("%d",&ch);
```

```
        switch(ch)
        {
                case 1: Insert();       break;
                case 2: Delete();       break;
                case 3: Disp();         break;
                case 4: return;
                default : printf("Wrong choice");
        }
 }while(ch!=4);
 getch();
 }

 void Insert()
 {
        if(rear==9)
                printf("Queue is full");
        else
        {
                printf("Enter a number to insert : ");
                scanf("%d",&num);
                if(front==-1)   // this is the first element
                        front++;
                rear++;
                queue[rear]=num;
                Disp();
        }
}

void Delete()
{
        if((front==rear==-1) || (front>rear))
                printf("Queue is empty");
        else
        {
                printf("Element %d is deleted\n",queue[front]);
```

```
                front++;
                Disp();
        }
    }

    void Disp()
    {
        if((front==rear==-1) || (front>rear))
                printf("Queue is empty");
        else
        {       printf("Queue : ");
                for(i=0; i<front; i++)
                        printf("__ ");
                for(i=front; i<=rear; i++)
                        printf("%5d",queue[i]);
        }
    }
```

OUTPUT:

1. Insert
2. Delete
3. Diplay
4. Exit

Enter choice: 1

Enter a number to insert: 21

Queue: 21

Enter choice: 2

Element 21 is deleted

Queue is empty

Enter choice: 3

Queue is empty

Enter choice: 4

CONCLUSION: In this way we have studied implementation of queue.

Experiment No. : 4

Implement Tower of Hanoi using recursion

AIM: Implement Tower of Hanoi using recursion.

OBJECTIVES: the concept of recursion is implemented

OUTCOMES: The students will be able to implement Tower of Hanoi problem using recursion

INPUT: Enter no. of disks

OUTPUT: The Tower of Hanoi involved moves

THEORY:

Tower of Hanoi, is a mathematical puzzle which consists of three tower (pegs) and more than one rings. These rings are of different sizes and stacked upon in ascending order i.e. the smaller one sits over the larger one. There are other variations of puzzle where the number of disks increase, but the tower count remains the same.

Tower of Hanoi, first we need to learn how to solve this problem with lesser amount of disks, say → 1 or 2. We mark three towers with name, source, destination and aux (only to help moving disks). If we have only one disk, then it can easily be moved from source to destination peg.

If we have 2 disks –

- First we move the smaller one (top) disk to aux peg
- Then we move the larger one (bottom) disk to destination peg
- And finally, we move the smaller one from aux to destination peg.

So now we are in a position to design algorithm for Tower of Hanoi with more than two disks. We divide the stack of disks in two parts. The largest disk (n^{th} disk) is in one part and all other (n-1) disks are in second part.Our ultimate aim is to move disk n from source to destination and then put all other (n-1) disks onto it.

Algorithm/Pseudo code:

START

Procedure Hanoi(disk, source, dest, aux)

IF disk == 0, THEN

move disk from source to dest

ELSE

```
        Hanoi(disk - 1, source, aux, dest)    // Step 1
        move disk from source to dest         // Step 2
        Hanoi(disk - 1, aux, dest, source)    // Step 3
      END IF

    END Procedure
    STOP
```

PROGRAM:

```
/* To implement tower of Honai using recursion */

#include "stdio.h"
void towers(int,char,char,char);

void towers(int n,char frompeg,char topeg,char auxpeg)
        { /* If only 1 disk, make the move and return */
         if(n==1)
          { printf("\nMove disk 1 from peg %c to peg %c",frompeg,topeg);
           return;
          }
         /* Move top n-1 disks from A to B, using C as auxiliary */
         towers(n-1,frompeg,auxpeg,topeg);
         /* Move remaining disks from A to C */
         printf("\nMove disk %d from peg %c to peg %c",n,frompeg,topeg);
         /* Move n-1 disks from B to C using A as auxiliary */
         towers(n-1,auxpeg,topeg,frompeg);
        }
void main()
        { int n;
         clrscr();
         printf("Enter the number of disks : ");
         scanf("%d",&n);
         printf("The Tower of Hanoi involves the moves :\n\n");
         towers(n,'A','C','B');
```

```
    getch();
}
```

OUTPUT:

Enter the number of disks : 3

The Tower of Hanoi involves the moves:

Move disk 1 from peg A to peg C
Move disk 2 from peg A to peg B
Move disk 1 from peg C to peg B
Move disk 3 from peg A to peg C
Move disk 1 from peg B to peg A
Move disk 2 from peg B to peg C
Move disk 1 from peg A to peg C

CONCLUSION: We have studied Tower of Hanoi.

Experiment No. : 5

Implement Sequential and Binary searching techniques

AIM: By applying the knowledge of searching we will implement Sequential and Binary searching.

OBJECTIVES: The fundamental of searching method id used.

OUTCOMES: The students will be able do searching.

THEORY:

Linear search is a very simple search algorithm. In this type of search, a sequential search is made over all items one by one. Every items is checked and if a match founds then that particular item is returned otherwise search continues till the end of the data collection.

Algorithm/Pseudo code:

Algorithm: To Search an element using linear search

Step 1. Set k := 1 & loc : = 0

Step 2. Repeat step 3 & 4 while loc : = 0 &k < = n

Step 3. If (item = data[k])

loc : = k

Else

K = k + 1

Step 4. If loc : = 0 ,then

Print "no. not found"

Else

Print "loc is the location of item"

Step 5. Exit

Algorithm: To Search an element using sequential search:

Input: List of size N. Target value T
Output: Position of T in the list I

BEGIN
Step 1. Set FOUND to false
Set I to 0

Step 2. While (I<=N) and (FOUND is false)
If List [I] = T
FOUND = true
Else
I=I+1
END

Step 3. If FOUND is false
T is not present in List.
END

PROGRAM:

```
/* Linear search in an unsorted array. */

#include <stdio.h>
#include <conio.h>
void main( )
{
    int arr[10] = { 11, 2, 9, 13, 57, 25, 17, 1, 90, 3 } ;
    int i, num ;
    clrscr( ) ;

    printf ( "Enter number to search: " ) ;
    scanf ( "%d", &num ) ;

    for ( i = 0 ; i <= 9 ; i++ )
```

```
    {
        if ( arr[i] == num )
            break ;
    }

    if ( i == 10 )
        printf ( "Number is not present in the array." ) ;
    else
        printf ( "The number is at position %d in the array.", i ) ;

    getch( ) ;
}
```

OUTPUT:

Enter number to search: 25

The number is at position 5 in the array. */

PROGRAM:

//C program to search a key number in an array using Sequential Search Method

```
#include<stdio.h>
#include<conio.h>
main()
{
    int arr[]={12,23,78,98,67,56,45,19,65,9},key,i,flag=0;
    clrscr();
    printf("\nENTER A NUMBER: ");
    scanf("%d",&key);
    for(i=0;i<10;i++)
    {
        if(key==arr[i])
            flag=1;
    }
    if(flag==1)
        printf("\nTHE NUMBER %d EXISTS IN THE ARRAY",key);
```

```
    else
            printf("\nTHE NUMBER %d DOES NOT EXIST IN THE ARRAY",key);
    getch();
}
```

CONCLUSION: We have studied and implementation of sequential and linear search.

Experiment No. : 6

Implement Quick sort

AIM: Implementation of Quick sort.

OBJECTIVES: To study the sorting methods using quick sort.

OUTCOMES: The students will be able tounderstand quick sort.

THEORY:

Quick sort:

Quick sort is a highly efficient sorting algorithm and is based on partitioning of array of data into smaller arrays. A large array is partitioned into two arrays one of which holds values smaller than specified value say pivot based on which the partition is made and another array holds values greater than pivot value. The quick sort partitions an array and then calls itself recursively twice to sort the resulting two sub array. This algorithm is quite efficient for large sized data sets as its average and worst case complexity are of O(nlogn) where n are no. of items.

Algorithm/Pseudo code:

Step 1. Choose the highest index value has pivot

Step 2. Take two variables to point left and right of the list excluding pivot

Step 3. left points to the low index

Step 4. right points to the high

Step 5. while value at left is less than pivot move right

Step 6. while value at right is greater than pivot move left

Step 7. if both step 5 and step 6 does not match swap left and right

Step 8. if left $\geq$ right, the point where they met is new pivot

PROGRAM:

```
/* Quick sort. */
#include <stdio.h>
#include <conio.h>
int split ( int*, int, int ) ;

void main( )
{
        int arr[10] = { 11, 2, 9, 13, 57, 25, 17, 1, 90, 3 } ;
        int i ;
        void quicksort ( int *, int, int ) ;
        clrscr( ) ;

        printf ( "Quick sort.\n" ) ;
        printf ( "\nArray before sorting:\n") ;

        for ( i = 0 ; i <= 9 ; i++ )
                printf ( "%d\t", arr[i] ) ;

        quicksort ( arr, 0, 9 ) ;

        printf ( "\nArray after sorting:\n") ;

        for ( i = 0 ; i <= 9 ; i++ )
                printf ( "%d\t", arr[i] ) ;

        getch( ) ;
}

void quicksort ( int a[ ], int lower, int upper )
{
        int i ;
        if ( upper > lower )
        {
                i = split ( a, lower, upper ) ;
```

```
            quicksort ( a, lower, i - 1 ) ;
            quicksort ( a, i + 1, upper ) ;
        }
}

int split ( int a[ ], int lower, int upper )
{
    int i, p, q, t ;

    p = lower + 1 ;
    q = upper ;
    i = a[lower] ;

    while ( q >= p )
    {
        while ( a[p] < i )
            p++ ;

        while ( a[q] > i )
            q-- ;

        if ( q > p )
        {
            t = a[p] ;
            a[p] = a[q] ;
            a[q] = t ;
        }
    }

    t = a[lower] ;
    a[lower] = a[q] ;
    a[q] = t ;

    return q ;
}
```

OUTPUT:

Quick sort

Array before sorting:

11 2 9 13 57 25 17 1 90 3

Array after sorting:

2 3 9 11 13 17 25 57 90

CONCLUSION: We have studied and implementation of quick sort.

Experiment No. : 7

Implement Merge sort

AIM: To Implement Merge sort

OBJECTIVES: Merge sort is implemented.

OUTCOMES: The students will be able to do merge sorting.

THEORY:

Merge sort:

Merge sort is a sorting technique based on divide and conquer technique. With worst-case time complexity being O(n log n), it is one of the most respected algorithms. Merge sort first divides the array into equal halves and then combines them in a sorted manner.

Algorithm/Pseudo code:

Step 1. If it is only one element in the list it is already sorted, return.

Step 2. Divide the list recursively into two halves until it can no more be divided.

Step 3. Merge the smaller lists into new list in sorted order.

PROGRAM: Merge sort

```
#include <stdio.h>
#include <conio.h>

void main( )
{
    int a[5] = { 11, 2, 9, 13, 57 } ;
    int b[5] = { 25, 17, 1, 90, 3 } ;
    int c[10] ;
    int i, j, k, temp ;

    clrscr( ) ;

    printf ( "Merge sort.\n" ) ;

    printf ( "\nFirst array:\n" ) ;
    for ( i = 0 ; i <= 4 ; i++ )
        printf ( "%d\t", a[i] ) ;
```

```
        printf ( "\n\nSecond array:\n" ) ;
        for ( i = 0 ; i <= 4 ; i++ )
                printf ( "%d\t", b[i] ) ;

        for ( i = 0 ; i <= 3 ; i++ )
        {
                for ( j = i + 1 ; j <= 4 ; j++ )
                {
                        if ( a[i] > a[j] )
                        {
                                temp = a[i] ;
                                a[i] = a[j] ;
                                a[j] = temp ;
                        }
                        if ( b[i] > b[j] )
                        {
                                temp = b[i] ;
                                b[i] = b[j] ;
                                b[j] = temp ;
                        }
                }
        }

        for ( i = j = k = 0 ; i <= 9 ; )
        {
                if ( a[j] <= b[k] )
                        c[i++] = a[j++] ;
                else
                        c[i++] = b[k++] ;

                if ( j == 5 || k == 5 )
                        break ;
        }
```

```
    for ( ; j <= 4 ; )
            c[i++] = a[j++] ;

    for ( ; k <= 4 ; )
            c[i++] = b[k++] ;

    printf ( "\n\nArray after sorting:\n") ;
    for ( i = 0 ; i <= 9 ; i++ )
            printf ( "%d\t", c[i] ) ;

    getch( ) ;
}
```

OUTPUT:

Merge sort.

First array:

11 2 9 13 57

Second array:

25 17 1 90 3

Array after sorting:

2 3 9 11 13 17 25 57 90

CONCLUSION: We have studied and implement merge sort.

Experiment No. : 8

Implement following operations on string

AIM: To implement string operations

a) String length

b) String copy

c) String reverse

d) String concatenation.

OBJECTIVES: Provide an understanding of different string operations.

OUTCOMES: The students will be able to understand string operations.

THEORY:

Strings are often needed to be manipulated by programmer according to the need of a problem. All string manipulation can be done manually by the programmer but, this makes programming complex and large. To solve this, the C supports a large number of string handling functions.

There are numerous functions defined in "string.h" header file. Few commonly used string handling functions are discussed below:

Function	Work of Function
strlen()	Calculates the length of string
strcpy()	Copies a string to another string
strcat()	Concatenates(joins) two strings
strcmp()	Compares two string
strlwr()	Converts string to lowercase
strupr()	Converts string to uppercase

PROGRAM:

```
/* To perform string operations */
#include<stdio.h>
#include<conio.h>
#include<string.h>
void main()
{
      char Fnm[15],Lnm[15],Fullnm[40];
      char word1[20],word2[20];
      int len;
      clrscr();
      printf("Enter your name : ");
      gets(Fnm);
      printf("Enter surename :");
      gets(Lnm);
      strcat(Fnm," ");      // append blank space to first name
      strcat(Fnm,Lnm);      // append last name to first name
      printf("\nFull name : %s",Fnm);
      printf("\n\n\nNow checking for pallindrome");
      printf("\nEnter word1 : ");
      gets(word1);
      strcpy(word2,word1);   //copy 'word1' into 'word2'
      strrev(word2);         // reverse the word2
```

```
    printf("\nGiven word = %s",word1);
    printf("\nReverse word = %s",word2);
    if(strcmp(word1,word2)==0)  // copmares both words
    printf("\n Given word is paildrome");
    else    printf("\n Given word is notpaildrome");
    getch();
}
```

OUTPUT:

Enter your name: vanashri

Enter surname: mali

Full name: vanashri mali

Now checking for pallindrome

Enter word1: nayan

Given word = nayan

Reverse word = nayan

Given word is palindrome

CONCLUSION: We have studied and implement different operations on string.

Experiment No. : 9

Implement following operations on file

AIM: To implement different operations on file

a) File read

b) File write

c) File copy

OBJECTIVES: To perform different operations on file.

OUTCOMES: The students will be able to understand file operations.

THEORY:

When the program is terminated, the entire data is lost in C programming. If you want to keep large volume of data, it is time consuming to enter the entire data. But, if file is created, these information can be accessed using few commands. There are large numbers of functions to handle file I/O in C language. In this tutorial, you will learn to handle standard I/O(High level file I/O functions) in C.

File Operations

- Opening an existing file

  Opening a file is performed using library function fopen().

- Reading from and writing information to a file

  Functions fread() and fwrite() are used for reading from and writing to a file on the disk respectively in case of binary files.Function fwrite() takes four arguments, address of data to be written in disk, size of data to be written in disk, number of such type of data and pointer to the file where you want to write.

- Closing a file

  The file should be closed after reading/writing of a file. Closing a file is performed using library function fclose().

PROGRAM:

```
/* Perform File Operation */

#include<stdio.h>
#include<stdlib.h>

void main()
{
/* 's' is source FILE pointer
   't' is target FILE pointer      */
FILE *s,*t;
char ch;
clrscr();

s=fopen("file1.txt","r");  //opens file 'file1.txt' in 'read' mode
if(s==NULL)
        printf("\nCan't open file1.txt");

t=fopen("file2.txt","w");  //opens file 'file2.txt' in 'write' mode
if(t==NULL)
        printf("\nCan't open file2.txt");

printf("\n File1.txt contains :\n\n\n");
while(1)
{
        ch=fgetc(s);   // gets a character from file1.txt
        if(ch==EOF)    // end of file,  all characters are processed
                break;
        else
        {     printf("%c",ch);     //disp. content of file
                fputc(ch,t);        // copy from file1 to file2
        }
}
printf("\n\n\nfile1.txt is copied in file2.txt");
fclose(s);
```

```
fclosse();

getch();
}
```

OUTPUT:

File1.txt contains :

Departmrnt of trchnology...

file1.txt is copied in file2.txt * /

CONCLUSION: We have studied different operations on file.

Experiment No. : 10

Implement circular queue using array

AIM: To study Implement circular queue using array

OBJECTIVES: To be able to discuss the Implement circular queue using array.

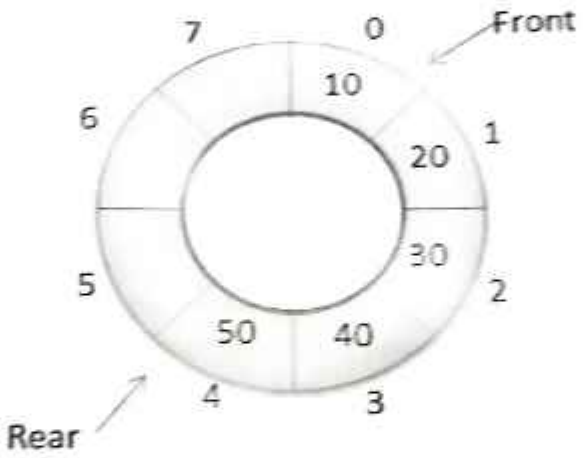
OUTCOMES: The students will be able to understand Implement circular queue using array.

THEORY:

Circular queue:

A circular queue is an abstract data type that contains a collection of data which allows addition of data at the end of the queue and removal of data at the beginning of the queue. Circular queues have a fixed size.

Circular queue follows FIFO principle. Queue items are added at the rear end and the items are deleted at front end of the circular queue.



Double Ended Queue

- DeQueue stands for Double Ended Queue.
- It is just like a queue but does not support FIFO structure.
- Insertion and deletion can be done from both side(FRONT & REAR).

Algorithm:

Step 1. Insertion inCircularQueue
Step 2. If (FRONT ==1and REAR == N)or(FRONT == REAR +1)Then
Step 3. Print:Overflow
Step 4. Else
Step 5. If(REAR ==0)Then[Checkif QUEUE is empty]
(a)Set FRONT =1
(b)Set REAR =1
Step 6. ElseIf(REAR == N)Then[If REAR reaches endif QUEUE]
Step 7. Set REAR =1
Step 8. Else
Step 9. Set REAR = REAR +1[Increment REAR by1]
[End of Step4If]
Step 10. Set QUEUE[REAR]= ITEM
Step 11. Print: ITEM inserted
[End of Step1If]
Step 12. Exit

Algorithm:

Deletion inCircularQueue()

Step 1. 1.If(FRONT ==0)Then[CheckforUnderflow]
Print:Underflow
Step 2. Else
ITEM = QUEUE[FRONT]
Step 3. If(FRONT == REAR)Then[If only element is left]
(a)Set FRONT =0
(b)Set REAR =0
Step 4. ElseIf(FRONT == N)Then[If FRONT reaches endif QUEUE]
Step 5. Set FRONT =1
Step 6. Else
Step 7. Set FRONT = FRONT +1[Increment FRONT by1]
[End of Step5If]
Step 8. 10.Print: ITEM deleted
[End of Step1If]
Step 9. Exit

PROGRAM:

```
/* Circular Queue */

#include<stdio.h>
void Insert();
void Delete();
void Disp();
int queue[5];
int front,rear,i,ch,num;

void main()
{
clrscr();
front=rear=-1;   // initially queue is empty
printf("1.Insert \n2.Delete \n3.Diplay \n4.Exit");
do
{
        printf("\n\nEnter choice : ");
        scanf("%d",&ch);
        switch(ch)
        {
                case 1: Insert();       break;
                case 2: Delete();       break;
                case 3: Disp();         break;
                case 4: return;
                default : printf("Wrong choice");
        }
}while(ch!=4);
getch();
}
void Insert()
{
      if((front==-1 && rear==4) || (front==rear+1))
              printf("Queue is full");
      else
```

```
        {
            printf("Enter a number to insert : ");
            scanf("%d",&num);
            if(front==-1)   // this is the first element
                front++;
            if(rear==4)
                rear=0;
            else
                rear++;
            queue[rear]=num;
            Disp();
        }
    }
    void Delete()
    {
        if(front==-1)
            printf("Queue is empty");
        else
        {
            printf("Element %d is deleted\n",queue[front]);
            queue[front]=0;
            if(front==9)
                front=0;
            else
                front++;
            Disp();
        }
    }
    void Disp()
    {
        if(front==-1)
            printf("Queue is empty");
        else
        {   printf("Queue : ");
            for(i=0; i<5; i++)
```

```
        {
            printf("%5d",queue[i]);
        }
    }
}
```

OUTPUT:

1. Insert
2. Delete
3. Diplay
4. Exit

Enter choice : 1
Enter a number to insert : 20
Queue : 20 0 0 0 0

Enter choice : 1
Enter a number to insert : 30
Queue : 20 30 0 0 0

Enter choice : 2
Element 20 is deleted
Queue : 0 30 0 0 0

Enter choice : 3
Queue : 0 30 0 0 0

Enter choice : 4

CONCLUSION: We have studied implementation of circular queue using array.

Experiment No. : 11

Implement palindrome of string using recursion and without recursion

AIM: To implement palindrome of string using recursion and without recursion.

OBJECTIVES: To implement palindrome of string using recursion and without recursion.

OUTCOMES: The students will be able to implement palindrome of string using recursion and without recursion.

THEORY:

A string is said to be palindrome if reverse of the string is same as string. For example, "abba" is palindrome, but "abbc" is not palindrome.

In the program first we copy the entered string into a new string, and then we reverse the new string and then compares it with original string. If both of them have same sequence of characters i.e. they are identical then the entered string is a palindrome otherwise not.

Algorithm:

Step 1: Input S (string)

Step 2: Len = 0 , Flag =0

Step 3: While (S[Len] != NULL)

Len++

Step 4: I = 0 , J = Len-1

Step 5: While (I < (Len/2)+1)

If (S[I] == S[J])

Flag=0

else

Flag=1

I++ , J–

Step 6: If (Flag == 0)

Print Key Is a Palindrome

else

Print Key Is Not a Palindrome

Step 7: End

PROGRAM:

```
/* To check given string is pallindrome or not */

#include<stdio.h>
#include<conio.h>
#include<string.h>

void main()
{
      char word1[20],word2[20];
      int num,n,a;
      long int revnum=0;
      clrscr();

     printf("\n Enter word1 : ");
     gets(word1);

    strcpy(word2,word1);    //copy 'word1' into 'word2'
    strrev(word2);          // reverse the word2

   printf("\n Given word = %s",word1);
   printf("\n Reverse word = %s",word2);

  if(strcmp(word1,word2)==0)  // copmares both words
         printf("\n Given word is paildrome");
  else
         printf("\n Given word is not paildrome");

 printf("\n\n Enter a 5 digit number (less than 32767) : ");
 scanf("%d",&num);

n=num;
a=n%10;        //5th digit
n=n/10;                  //remaining digits
revnum=revnum+(a*10000L);
```

```
    a=n%10;         //4th digit
    n=n/10;                 //remaining digits
    revnum=revnum+(a*1000);

    a=n%10;         //3rd digit
    n=n/10;                 //remaining digits
    revnum=revnum+(a*100);

    a=n%10;         //2nd digit
    n=n/10;                 //remaining digits
    revnum=revnum+(a*10);

    a=n%10;                 //1st digit
    revnum=revnum+a;

    printf("\n\n Given number is : %d",num);
    printf("\n Reverse number is : %ld",revnum);

   if(num==revnum)
        printf("\n Number is palindrome");
  else  printf("\n Number is not pallindrome");

   getch();
}
```

OUTPUT:

Enter word1 : nayan

Given word = nayan

Reverse word = nayan

Given word is paildrome

Enter a 5 digit number (less than 32767) : 12321

Given number is : 12321

Reverse number is : 12321

Number is palindrome

CONCLUSION: In this way we have studied Palindrome string.

Experiment No. : 12

Implement menu driven program to perform below operations on singly linked list

AIM: To implement menu driven program to perform below operations on singly linked list.

- Create
- Insert.
- Delete.
- Display.

OBJECTIVES: To implement menu driven program to perform below operations on singly linked list.

OUTCOMES: The students will be able implement Write menu driven program to perform below operations on singly linked list.

THEORY:

A linked-list is a sequence of data structures which are connected together via links. Linked List is a sequence of links which contains items. Each link contains a connection to another link. Linked list the second most used data structure after array. Following are important terms to understand the concepts of Linked List.

- **Link** – Each Link of a linked list can store a data called an element.
- **Next** – Each Link of a linked list contain a link to next link called Next.
- **LinkedList** – A LinkedList contains the connection link to the first Link called First.

Following are the various flavours of linked list.

- **Simple Linked List** – Item Navigation is forward only.
- **Doubly Linked List** – Items can be navigated forward and backward way.
- **Circular Linked List** – Last item contains link of the first element as next and and first element has link to last element as prev.

Following are the basic operations supported by a linked list.

- **Insertion** – add an element at the beginning of the list.
- **Deletion** – delete an element at the beginning of the list.
- **Display** – displaying complete list.
- **Search** – search an element using given key.
- ing given key.

PROGRAM:

```
#include<stdio.h>
#include<alloc.h>
voidaddAtBegt();
voidaddAtEnd();
voidaddAfter();
void Delete();
voidDisp();

struct node
{
        int data;
        struct node *prev,*next;
};

struct node *start,*q,*temp,*old;
intnum,loc,i,ch;

void main()
{
        start=NULL;  // initially list is empty
        clrscr();

        printf("1.Add at begin \n2.Add at end \n3.Add after \n4.Delete \n5.Display \n6.Exit");
        do
        {
                printf("\n\nEnter choice : ");
                scanf("%d",&ch);
                switch(ch)
                {
```

```
            case 5: Disp();          break;
            case 6: return;
            default :printf("\nWrong choice");
            }
        }while(ch!=6);
getch();
}

voidaddAtBeg()
{
    q=start;
    printf("Enter no to add at begining: ");
    scanf("%d",&num);

    temp=malloc(sizeof(struct node));
    temp->data=num;
    temp->prev=NULL;      // 'prev' ptr. of first node is always NULL
    temp->next=q;
    if(start!=NULL)
        q->prev=temp;

   start=temp;     // 'start' points to the first node
   Disp();
}

voidaddAtEnd()
{
    printf("Enter no to add at end: ");
    scanf("%d",&num);

   if(start==NULL)     //if list is empty
   {
        temp=malloc(sizeof(struct node));
        temp->data=num;
        t=NULL;  // link part of last node is always NULL
```

```
            temp->prev=NULL;
            start=temp;      // this is the first node
        }
        else
        {
            q=start;
            while(q->next!=NULL)  //traverse untill we get last node
                q=q->next;

            temp=malloc(sizeof(struct node));
            temp->data=num;
            temp->next=NULL;
            temp->prev=q;   // link part of last node is always NULL
            q->next=temp;      // append new node at the end of last node
        }
        Disp();
}

voidaddAfter()
{
     printf("Enter no to add:");
     scanf("%d",&num);
     printf("Enter loc after which u want to add: ");
     scanf("%d",&loc);

    q=start;
    for(i=1;i<loc;i++)
    {
        q=q->next;
        if(q==NULL)
        {
            printf("There are less than %d numbers\n",loc);
            return;
        }
    }
```

```
        temp=malloc(sizeof(struct node));
        temp->data=num;
        temp->next=q->next;
        temp->prev=q;
        if(q->next!=NULL)
        {
                old=q;
                q=q->next;
                q->prev=temp;
                old->next=temp;
        }
        else    q->next=temp;
        Disp();
}

void Delete()
{
    if(start==NULL)
    {
            printf("List is empty\n");
            return;
    }
    printf("Enter no to delete : ");
    scanf("%d",&num);

   q=start;
   while(q!=NULL)
   {
           if(q->data==num)
           {
                   if(q==start)
                   {
                           start=q->next;
                           start->prev=NULL;
                   }
```

```
                else
                {
                        old->next=q->next;
                        temp=old->next;
                }

                free(q);
                temp->prev=old;
                Disp();
                return;
            }
            else
            {
                old=q;
                q=q->next;
            }
        }
        printf("Element not found\n");
}

voidDisp()
{
     printf("List :\n");
     q=start;
     if(q==NULL)
          printf(" Empty\n");
     else
     {
          while(q!=NULL)
          {     printf("%5d",q->data);
                q=q->next;
          }

     }
}
```

OUTPUT:

```
Enter choice: 1
Enter no to add at beg: 10

Enter choice: 2
Enter no to add at end: 20
List:
 10  20

Enter choice: 3
Enter no to add: 30
Enter loc after which u want to add: 2
List:
 10  20  30

Enter choice: 4
Enter no to delete: 30
List:
 10  20

Enter choice: 5
List:
 10  20

Enter choice: 6
Null pointer assignment
```

CONCLUSION:

In this way we have studied performing different operations on linked list.

Experiment No. : 13

To implement of binary tree for insertion, in order traversal operation

AIM: To implement of binary tree for below operation, insertion, in order traversal.

OBJECTIVES: To implement of binary tree for below operation;

- insertion
- in order traversal

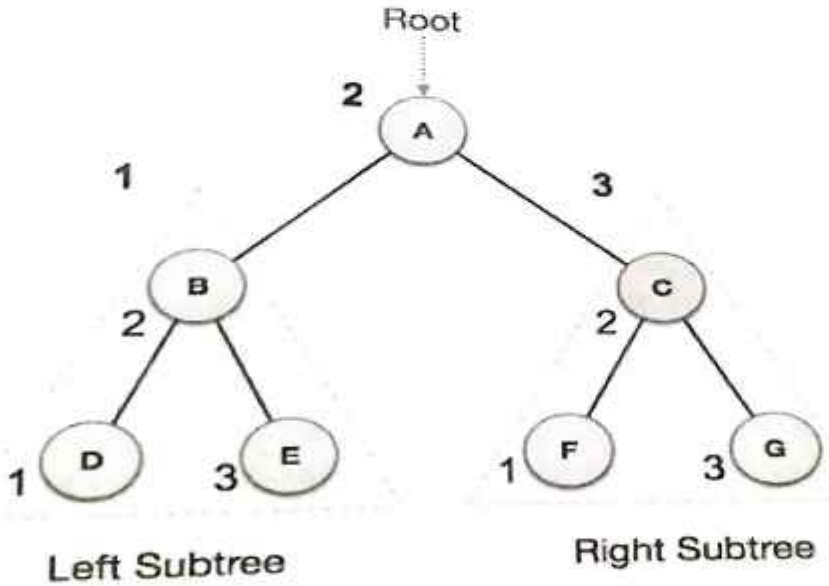
OUTCOMES: The students will be able to implement of binary tree for below operation; insertion, in order traversal.

THEORY:

Traversal is a process to visit all the nodes of a tree and may print their values too. Because, all nodes are connected via edges (links) we always start from the root (head) node. That is, we cannot random access a node in tree. There are three ways which we use to traverse a tree.

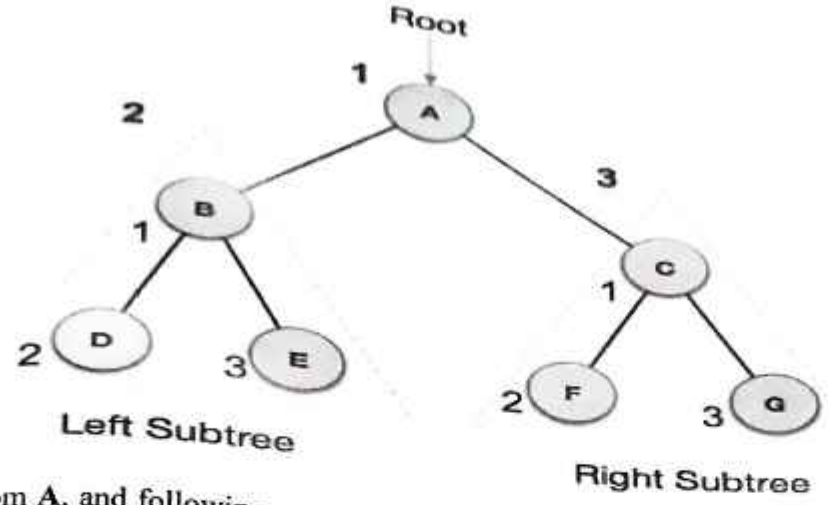
In-order Traversal:

In this traversal method, the left left-subtree is visited first, then root and then the right sub-tree. We should always remember that every node may represent a subtree itself.If a binary tree is traversed **inorder**, the output will produce sorted key values in ascending order.



We start fro[illegible]owing in-order traversal, we move to its left subtree**B**. **B** is also travers[illegible]ntil all the nodes are visited. The output of in-

In this traversal method, the root node is visited first, then left subtree and finally right sub-tree.

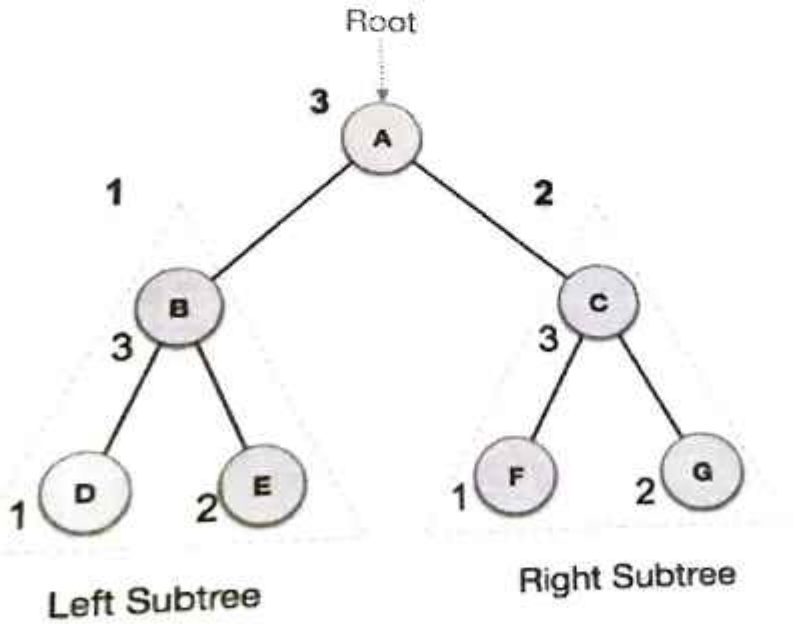


We start from **A**, and following pre-order traversal, we first visit **A** itself and then move to its left subtree**B**. **B** is also traversed pre-ordered. And the process goes on until all the nodes are visited. The output of pre-order traversal of this tree will be –

$$A \rightarrow B \rightarrow D \rightarrow E \rightarrow C \rightarrow F \rightarrow G$$

***Post-order* Traversal:**

In this traversal method, the root node is visited last, hence the name. First we traverse left subtree, then right subtree and finally root.



We start from **A**, and following pre-order traversal, we first visit left subtree**B**. **B** is also traversed pre-ordered. And the process goes on until all the nodes are visited. The output of pre-order traversal of this tree will be –

$\rightarrow E \rightarrow B \rightarrow F \rightarrow G \rightarrow C \rightarrow$

PROGRAM:

```
/* Program to implement a binary search tree traversal. */

#include <stdio.h>
#include <conio.h>
#include <alloc.h>
struct node
{
        struct node *left;
        int data;
        struct node *right;
};

void insert(struct node **, int);
voidinorder(struct node *);
void preorder(struct node *);
voidpostorder(struct node *);

void main()
{
      struct node *bt;
      intreq, i=1, num;

     bt=NULL;  /* empty tree */

     clrscr();
     printf("Specify the number of items to be inserted: ");
     scanf("%d",&req);

    while(i<=req)
    {
          printf("Enter the data: ");
```

```
            scanf("%d", &num);
            insert(&bt, num);
            i++;
        }

        printf("\nIn-order  Traversal: ");
        inorder(bt);

        printf("\nPre-order  Traversal: ");
        preorder(bt);

        printf("\nPost-order Traversal: ");
        postorder(bt);
        getch();
}

/* inserts a new node in a binary search tree */
void insert(struct node **sr, intnum)
{
    if(*sr==NULL)
    {
        *sr=malloc(sizeof(struct node));

        (*sr)->left=NULL;
        (*sr)->data=num;
        (*sr)->right=NULL;
        return;
    }
    else  /* search the node to which new node will be attached */
    {
        /* if new data is less, traverse to left */
        if(num<(*sr)->data)
            insert(&((*sr)->left), num);
        else
            /* else traverse to right */
```

```
                insert(&((*sr)->right), num );
        }
        return;
}
/* traverse a binary search tree in a LDR (Left-Data-Right) fashion */
voidinorder(struct node *sr)
{
    if(sr!=NULL)
    {
        inorder(sr->left);

        /* print the data of the node whose left is NULL or the path
        has already been traversed */
        printf("\t%d",sr->data);

        inorder(sr->right);
    }
    else
        return;
}

/* traverse a binary search tree in a DLR (Data-Left-right) fashion */
void preorder(struct node *sr)
{
    if(sr!= NULL)
    {
        /* print the data of a node */
        printf("\t%d",sr->data);
        /* traverse till left is not NULL */
        preorder(sr->left);
        /* traverse till right is not NULL */
        preorder(sr->right);
    }
    else
```

```
        return;
}

/* traverse a binary search tree in LRD (Left-Right-Data) fashion */
void postorder ( struct node *sr )
{
    if ( sr != NULL )
    {
        postorder ( sr -> left ) ;
        postorder ( sr -> right ) ;
        printf ( "\t%d", sr -> data ) ;
    }
    else
        return;
}
```

OUTPUT:

```
Specify the number of items to be inserted: 4
Enter the data: 56
Enter the data: 34
Enter the data: 87
Enter the data: 21

In-order  Traversal:  21   34   56   87
Pre-order  Traversal:  56   34   21   87
Post-order Traversal:  21   34   87   56   */
```

CONCLUSION: In this way we have studied different tree traversal methods.

Experiment No. : 14

To study graph traversal method

AIM: To study graph traversal method.

OBJECTIVES: To study graph traversal method.

OUTCOMES: The students will be able to study graph traversal method.

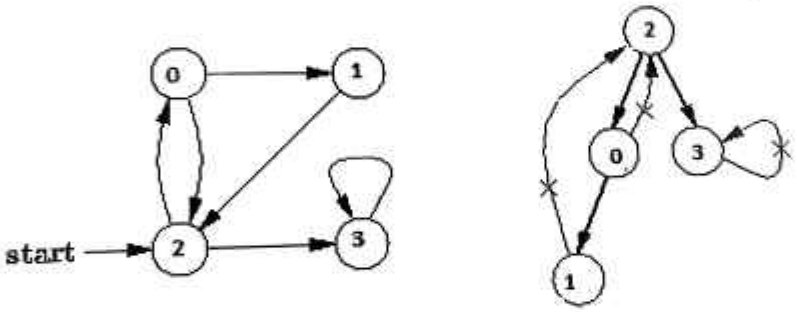
- insertion
- in order traversal

THEORY:

Two standard graph search techniques have been widely used:

Depth-First Search (DFS)

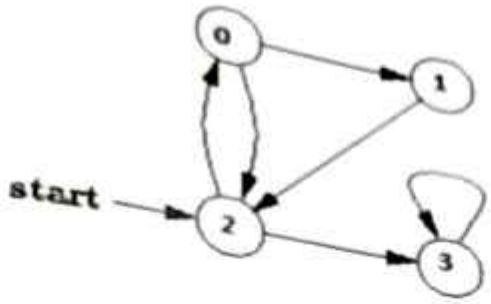
Depth First Traversal (or Search) for a graph is similar to Depth First Traversal of a tree. The only catch here is, unlike trees, graphs may contain cycles, so we may come to the same node again. To avoid processing a node more than once, we use a Boolean visited array. For example, in the following graph, we start traversal from vertex 2. When we come to vertex 0, we look for all adjacent vertices of it. 2 is also an adjacent vertex of 0. If we don't mark visited vertices, then 2 will be processed again and it will become a non-terminating process. A Depth First Traversal of the following graph is 2, 0, 1, 3.



Breadth-First Search (BFS)

Breadth First Traversal (or Search) for a graph is similar to Breadth First Traversal of a tree (See method 2 of this post). The only catch here is, unlike trees, graphs may contain cycles, so we may come to the same node again. To avoid processing a node more than once, we use a boolean visited array. For simplicity, it is assumed that all vertices are reachable from the starting vertex.

For example, in the following graph, we start traversal from vertex 2. When we come to vertex 0, we look for all adjacent vertices of it. 2 is also an adjacent vertex of 0. If we don't mark visited vertices, then 2 will be processed again and it will become a non-terminating process. A Breadth First Traversal of the following graph is 2, 0, 3, 1.



PROGRAM:

```
// C++ program to print DFS traversal from
// a given vertex in a  given graph
#include<iostream>
#include<list>
using namespace std;

// Graph class represents a directed graph
// using adjacency list representation
class Graph
{
    int V;    // No. of vertices

    // Pointer to an array containing
    // adjacency lists
    list<int> *adj;

    // A recursive function used by DFS
    void DFSUtil(int v, bool visited[]);
public:
    Graph(int V);  // Constructor

    // function to add an edge to graph
    void addE
```

```
        // DFS traversal of the vertices
        // reachable from v
        void DFS(int v);
};

Graph::Graph(int V)
{
        this->V = V;
        adj = new list<int>[V];
}

void Graph::addEdge(int v, int w)
{
        adj[v].push_back(w); // Add w to v's list.
}

void Graph::DFSUtil(int v, bool visited[])
{
        // Mark the current node as visited and
        // print it
        visited[v] = true;
        cout << v << " ";

        // Recur for all the vertices adjacent
        // to this vertex
        list<int>::iterator i;
        for (i = adj[v].begin(); i != adj[v].end(); ++i)
          if (!visited[*i])
            DFSUtil(*i, visited);
}

// DFS traversal of the vertices reachable from v.
// It uses recursive DFSUtil()
```

```
void Graph::DFS(int v)
{
    // Mark all the vertices as not visited
    bool *visited = new bool[V];
    for (int i = 0; i < V; i++)
      visited[i] = false;

    // Call the recursive helper function
    // to print DFS traversal
    DFSUtil(v, visited);
}

int main()
{
    // Create a graph given in the above diagram
    Graph g(4);
    g.addEdge(0, 1);
    g.addEdge(0, 2);
    g.addEdge(1, 2);
    g.addEdge(2, 0);
    g.addEdge(2, 3);
    g.addEdge(3, 3);
    cout << "Following is Depth First Traversal"
         " (starting from vertex 2) \n";
    g.DFS(2);
    return 0;
}
```

Following is Depth First Traversal (starting from vertex 2)

2 0 1 3

CONCLUSION: In this way we have studied study graph traversal method.

Experiment No. : 15

To study tree traversal method

AIM: To study tree traversal method.

OBJECTIVES: To study tree traversal method.

OUTCOMES: The students will be able to study tree traversal method.

- insertion
- in order traversal

THEORY:

Traversing a tree involves iterating over all nodes in some manner. Because from a given node there is more than one possible next node (it is not a linear data structure), then, assuming sequential computation (not parallel), some nodes must be deferred—stored in some way for later visiting. This is often done via a stack (LIFO) or queue (FIFO). As a tree is a self-referential (recursively defined) data structure, traversal can be defined by recursion or, more subtly, corecursion, in a very natural and clear fashion; in these cases the deferred nodes are stored implicitly in the call stack.

Depth-first search is easily implemented via a stack, including recursively (via the call stack), while breadth-first search is easily implemented via a queue, including corecursively.

Depth-first search

These searches are referred to as depth-first search (DFS), as the search tree is deepened as much as possible on each child before going to the next sibling. For a binary tree, they are defined as display operations recursively at each node, starting with the root, whose algorithm is as follows.

The general recursive pattern for traversing a (non-empty) binary tree is this: At node N do the following:

(L) Recursively traverse its left subtree. This step is finished at the node N again.

(R) Recursively traverse its right subtree. This step is finished at the node N again.

(N) Process N itself.

These steps can be done in any order. If (L) is done before (R), the process is called left-to-right traversal, otherwise it is called right-to-left traversal. The following methods show left-to-right traversal:

Pre-order traversal while duplicating nodes and edges can make a complete duplicate of a binary tree. It can also be used to make a prefix expression (Polish notation) from expression trees: traverse the expression tree pre-orderly.

In-order traversal is very commonly used on binary search trees because it returns values from the underlying set in order, according to the comparator that set up the binary search tree (hence the name).

Post-order traversal while deleting or freeing nodes and values can delete or free an entire binary tree. It can also generate a postfix representation of a binary tree.

Inorder Traversal

Algorithm Inorder(tree)

1. Traverse the left subtree, i.e., call Inorder(left-subtree)
2. Visit the root.
3. Traverse the right subtree, i.e., call Inorder(right-subtree)

Uses of Inorder

In case of binary search trees (BST), Inorder traversal gives nodes in non-decreasing order. To get nodes of BST in non-increasing order, a variation of Inorder traversal where Inorder traversal s reversed can be used.

Example: Inorder traversal for the above-given figure is 4 2 5 1 3.

Preorder Traversal:

Algorithm Preorder(tree)

1. Visit the root.
2. Traverse the left subtree, i.e., call Preorder(left-subtree)
3. Traverse the right subtree, i.e., call Preorder(right-subtree)

Postorder Traversal:

Algorithm Postorder(tree)

1. Traverse the left subtree, i.e., call Postorder(left-subtree)
2. Traverse the right subtree, i.e., call Postorder(right-subtree)
3. Visit the root.

Uses of Postorder

Postorder traversal is used to delete the tree Postorder traversal is also useful to get the postfix expression of an expression tree.

Example: Postorder traversal for the above given figure is 4 5 2 3 1.

PROGRAM:

```
// C program for different tree traversals
#include <iostream>
using namespace std;

/* A binary tree node has data, pointer to left child
and a pointer to right child */
struct Node
{
        int data;
        struct Node* left, *right;
        Node(int data)
        {
                this->data = data;
                left = right = NULL;
        }
};

/* Given a binary tree, print its nodes according to the
"bottom-up" postorder traversal. */
void printPostorder(struct Node* node)
{
```

```
    if (node == NULL)
        return;

    // first recur on left subtree
    printPostorder(node->left);

    // then recur on right subtree
    printPostorder(node->right);

    // now deal with the node
    cout << node->data << " ";
}

/* Given a binary tree, print its nodes in inorder*/
void printInorder(struct Node* node)
{
    if (node == NULL)
        return;

    /* first recur on left child */
    printInorder(node->left);

    /* then print the data of node */
    cout << node->data << " ";

    /* now recur on right child */
    printInorder(node->right);
}

/* Given a binary tree, print its nodes in preorder*/
void printPreorder(struct Node* node)
{
    if (node == NULL)
        return;
```

```
        /* first print data of node */
        cout << node->data << " ";

        /* then recur on left sutree */
        printPreorder(node->left);

        /* now recur on right subtree */
        printPreorder(node->right);
}

/* Driver program to test above functions*/
int main()
{
        struct Node *root = new Node(1);
        root->left        = new Node(2);
        root->right       = new Node(3);
        root->left->left  = new Node(4);
        root->left->right = new Node(5);

        cout << "\nPreorder traversal of binary tree is \n";
        printPreorder(root);

        cout << "\nInorder traversal of binary tree is \n";
        printInorder(root);

        cout << "\nPostorder traversal of binary tree is \n";
        printPostorder(root);

        return 0;
}
```

OUTPUT:

Preorder traversal of binary tree is

1 2 4 5 3

Inorder traversal of binary tree is

4 2 5 1 3

Postorder traversal of binary tree is

4 5 2 3 1

CONCLUSION: In this way we have studied study graph traversal method.

List of Experiments

Sr. No.	Name of Experiment
1.	Write a program for matrix Manipulation using array.
2.	Implement Stack using array.
3.	Implement linear queue using array.
4.	Implement Tower of Hanoi using recursion.
5.	Implement Sequential and Binary searching techniques.
6.	Implement Quick sort.
7.	Implement Merge sort.
8.	Implement following operations on string a) String length b) String copy c) String reverse d) String concatenation.
9.	Implement following operations on file
10.	Implement circular queue using array
11.	Implement palindrome of string using recursion and without recursion.
12.	Write menu driven program to perform below operations on singly linked list.
13.	To implement of binary tree for below operation; insertion , in order traversal.
14.	To study graph traversal method.
15.	To study tree traversal method.